

2. RELATIONAL SCHEMA

Fan(email, name, phone)

Venue(address, name, capacity, contact_info)

Management(license_no, name)

Artist(license_no, artist_name, industry)

- license_no is a foreign key referencing Management(license_no)

Tour_Event(tour_name, event_date, license_no, artist_name)

- (license_no, artist_name) is a foreign key referencing Artist(license_no, artist_name)

Concert(tour_name, event_date, license_no, artist_name, setlist)

- (tour_name, event_date, license_no, artist_name) is a foreign key referencing Tour_Event(tour_name, event_date, license_no, artist_name)

Fansign(tour_name, event_date, license_no, artist_name, interaction_time)

- (tour_name, event_date, license_no, artist_name) is a foreign key referencing Tour_Event(tour_name, event_date, license_no, artist_name)

Listing(license_no, artist_name, tour_name, event_date, venue_address)

- (tour_name, event_date, license_no, artist_name) is a foreign key referencing Tour_Event(tour_name, event_date, license_no, artist_name)
- venue_address is a foreign key referencing Venue(address)

Likes(fan_email, license_no, artist_name)

- fan_email is a foreign key referencing Fan(email)
- (license_no, artist_name) is a foreign key referencing Artist(license_no, artist_name)

Employee(employee_id, venue_address, name)

- venue_address is a foreign key referencing Venue(address)

Ticket(license_no, artist_name, tour_name, event_date, venue_address, section, seat_no, price, tier)

- (license_no, artist_name, tour_name, event_date, venue_address) is a foreign key referencing Listing(license_no, artist_name, tour_name, event_date, venue_address)

Buys_Ticket(fan_email, employee_id, license_no, artist_name, tour_name, event_date, venue_address, section, seat_no, purchase_time, purchase_method)

- fan_email is a foreign key referencing Fan(email)
- employee_id is a foreign key referencing Employee(employee_id)
- (license_no, artist_name, tour_name, event_date, venue_address, section, seat_no) is a foreign key referencing Ticket(license_no, artist_name, tour_name, event_date, venue_address, section, seat_no)

Ticket_Price_Audit (audit_id, license_no, artist_name, tour_name, event_date, venue_address, section, seat_no, old_price, new_price, changed_by_employee_id, changed_at)

- Audit_id is the primary key
- (license_no, artist_name, tour_name, event_date, venue_address, section, seat_no) is a foreign key referencing Ticket(license_no, artist_name, tour_name, event_date, venue_address, section, seat_no)
- changed_by_employee_id is a foreign key referencing Employee(employee_id)

3. STORED PROCEDURE

(a)

This stored procedure implements dynamic pricing for available tickets. It takes an event date, a percentage increase, and a maximum allowed ticket price as input. It uses a cursor to iterate over all tickets in the `Ticket` table for events on the given date that have not been purchased (identified as tickets with no matching tuple in `Buys\_Ticket`). For each such ticket, it computes a new price by increasing the old price by the given percentage. If the computed price exceeds the specified cap, the procedure sets the price to the cap. It then updates the ticket price in the `Ticket` table.

(b)

(code will be inserted `q3\_procedure.sql`)

(c)

```
cs421g111@winter2026-comp421:~$ db2 DROP PROCEDURE Adjust_Ticket_Prices;
DB21034E The command was processed as an SQL statement because it was not a
valid Command Line Processor command. During SQL processing it returned:
SQL1024N A database connection does not exist.  SQLSTATE=08003
cs421g111@winter2026-comp421:~$ db2 connect to COMP421
```

Database Connection Information

```
Database server      = DB2/LINUX8664 11.5.9.0
SQL authorization ID = CS421G11...
Local database alias = COMP421
```

```
cs421g111@winter2026-comp421:~$ db2 -td@ -vf q3_procedure.sql
CREATE PROCEDURE Adjust_Ticket_Prices
(
    IN p_event_date DATE,
    IN p_percent_increase DECIMAL(5,2),
    IN p_price_cap DECIMAL(10,2)
)
LANGUAGE SQL
BEGIN
    DECLARE v_license_no VARCHAR(15);
    DECLARE v_artist_name VARCHAR(50);
    DECLARE v_tour_name VARCHAR(50);
    DECLARE v_event_date DATE;
    DECLARE v_venue_address VARCHAR(125);
    DECLARE v_section VARCHAR(10);
    DECLARE v_seat_no INTEGER;
```

```
DECLARE v_old_price DECIMAL(10,2);
DECLARE v_new_price DECIMAL(10,2);
DECLARE at_end INT DEFAULT 0;

DECLARE not_found CONDITION FOR SQLSTATE '02000';

DECLARE ticket_cursor CURSOR FOR
    SELECT T.license_no,
           T.artist_name,
           T.tour_name,
           T.event_date,
           T.venue_address,
           T.section,
           T.seat_no,
           T.price
    FROM Ticket T
    LEFT JOIN Buys_Ticket B
      ON T.license_no = B.license_no
     AND T.artist_name = B.artist_name
     AND T.tour_name = B.tour_name
     AND T.event_date = B.event_date
     AND T.venue_address = B.venue_address
     AND T.section = B.section
     AND T.seat_no = B.seat_no
   WHERE T.event_date = p_event_date
     AND B.fan_email IS NULL
     AND T.price IS NOT NULL;

DECLARE CONTINUE HANDLER FOR not_found
    SET at_end = 1;

OPEN ticket_cursor;
```

```
FETCH ticket_cursor INTO
    v_license_no,
    v_artist_name,
    v_tour_name,
    v_event_date,
    v_venue_address,
    v_section,
    v_seat_no,
    v_old_price;

WHILE at_end = 0 DO
    SET v_new_price = DECIMAL(DOUBLE(v_old_price) * (1 + DOUBLE(p_percent_increase) / 100), 10, 2);

    IF v_new_price > p_price_cap THEN
        SET v_new_price = p_price_cap;
    END IF;

    UPDATE Ticket
    SET price = v_new_price
    WHERE license_no = v_license_no
      AND artist_name = v_artist_name
      AND tour_name = v_tour_name
      AND event_date = v_event_date
      AND venue_address = v_venue_address
      AND section = v_section
      AND seat_no = v_seat_no;

    FETCH ticket_cursor INTO
        v_license_no,
        v_artist_name,
        v_tour_name,
        v_event_date,
        v_venue_address,
```

```

        FETCH ticket_cursor INTO
            v_license_no,
            v_artist_name,
            v_tour_name,
            v_event_date,
            v_venue_address,
            v_section,
            v_seat_no,
            v_old_price;
    END WHILE;

    CLOSE ticket_cursor;
END

DB21034E The command was processed as an SQL statement because it was not a
valid Command Line Processor command. During SQL processing it returned:
SQL0454N The signature provided in the definition for routine
"CS421G111.ADJUST_TICKET_PRICES" matches the signature of some other routine.
LINE NUMBER=88. SQLSTATE=42723

cs421g111@winter2026-comp421:~$

```

(d)

Code to look look at Tickets unsold on a particular day (2025-09-14):

```

SELECT T.license_no,
       T.artist_name,
       T.tour_name,
       T.event_date,
       T.section,
       T.seat_no,
       T.price
FROM Ticket T
LEFT JOIN Buys_Ticket B
  ON T.license_no = B.license_no
 AND T.artist_name = B.artist_name
 AND T.tour_name = B.tour_name
 AND T.event_date = B.event_date
 AND T.venue_address = B.venue_address
 AND T.section = B.section
 AND T.seat_no = B.seat_no
WHERE T.event_date = '2025-09-14'
   AND B.fan_email IS NULL
ORDER BY T.section, T.seat_no;

```

Code to check bought Tickets on a particular day (2025-09-14):

```

SELECT * FROM Buys_Ticket WHERE event_date = '2025-09-14';

```

Before:**Unsold tickets:**

LICENSE_NO	ARTIST_NAME	TOUR_NAME	EVENT_DATE	SECTION	SEAT_NO	PRICE
7264028809	Y-find	Blood & Chrome	09/14/2025	13	42	848.00
7264028809	Y-find	Blood & Chrome	09/14/2025	27	187	802.00
7264028809	Y-find	Blood & Chrome	09/14/2025	6	169	686.00

3 record(s) selected.

Sold tickets:

FAN_EMAIL	EMPLOYEE_ID	LICENSE_NO	ARTIST_NAME	SECTION	SEAT_NO	PURCHASE_TIME	TOUR_NAME	PURCHASE_METHOD	EVENT_DATE	VENUE_ADDRESS
cvedenyapin3x@mysql.com	13	7264028809	Y-find	63	94	2024-10-15-10.04.00.000000	Blood & Chrome	Credit	09/14/2025	993 Declaration Junction
garonsohn41@parallels.com	1	7264028809	Y-find	43	38	2025-01-11-10.26.00.000000	Blood & Chrome	Credit	09/14/2025	993 Declaration Junction
dcunrado3p@smh.com.au	16	8360087317	Zathin	90	15	2024-11-26-01.54.00.000000	Iron Throne	Cash	09/14/2025	1 Veith Junction
chowe44@home.pl	13	8360087317	Zathin	196	62	2024-10-29-04.46.00.000000	Iron Throne	Credit	09/14/2025	1 Veith Junction
mslocomb2z@bizjournals.com	23	7264028809	Y-find	41	126	2024-10-28-13.41.00.000000	Blood & Chrome	Cash	09/14/2025	993 Declaration Junction
edaileyz@utexas.edu	5	8360087317	Zathin	220	58	2024-10-11-09.18.00.000000	Iron Throne	Debit	09/14/2025	1 Veith Junction

6 record(s) selected.

Run code:

```
CALL Adjust_Ticket_Prices('2025-09-14', 10.00, 900.00);
```

```
db2 => CALL Adjust_Ticket_Prices('2025-09-14', 10.00, 900.00);

Return Status = 0
```

After:**Unsold tickets:**

- Note: all ticket price has increased and the first ticket has hit max (900.00)

LICENSE_NO	ARTIST_NAME	TOUR_NAME	EVENT_DATE	SECTION	SEAT_NO	PRICE
7264028809	Y-find	Blood & Chrome	09/14/2025	13	42	900.00
7264028809	Y-find	Blood & Chrome	09/14/2025	27	187	882.20
7264028809	Y-find	Blood & Chrome	09/14/2025	6	169	754.60

3 record(s) selected.

Sold tickets:

- Note: the tickets price has not changed (shouldn't been changed)

Section 4: Employee Menu

Landing Page

```
TiQ Main Menu
    1. Employees
    2. Fans
    0. Exit
Please Enter Your Option: 1
Employee ID: 5

=====
          TiQ Employee Menu
=====
    1. Add a new event
    2. Cancel an event
    3. Modify ticket prices
    0. Back
=====
```

When logging into the Employee menu, the user is first prompted to enter their Employee ID. This menu presents a simple console menu with three options: Add a new event, Cancel an event, and Modify ticket prices, plus a Back option to return to the main menu. The menu is displayed in a loop so the employee is always returned to it after completing any action.

Add a New Event

Successful Insert:

```

=====
      TiQ Employee Menu
=====
1. Add a new event
2. Cancel an event
3. Modify ticket prices
0. Back
=====
Enter your choice: 1

--- Add a New Event ---
Tour name: Electric Funeral
Event date (YYYY-MM-DD): 2026-04-10
Management license number: 5848943849
Artist name: Fix San
Event type - enter C for Concert, F for Fansign: F
Interaction time (minutes): 10
Event added successfully!

```

The employee is prompted to enter the tour name, event date, management license number, and artist name. After entering the event type as F for Fansign, they are prompted for the interaction time in minutes. Behind the scenes, the application first runs a `SELECT` query against the `Artist` table to verify that the provided license number and artist name correspond to an existing artist. If the artist is found, the application inserts a row into `Tour_Event` with the four identifying fields, and also inserts into the `Fansign` table with the interaction time. The user sees "Event added successfully!" as confirmation.

Error Example:

```

=====
      TiQ Employee Menu
=====
1. Add a new event
2. Cancel an event
3. Modify ticket prices
0. Back
=====
Enter your choice: 1

--- Add a New Event ---
Tour name: Faux Event
Event date (YYYY-MM-DD): 2026-12-12
Management license number: 123456
Artist name: John Doe
Error: No artist found with that license number and name. Event not added.

```

When the employee enters a license number and artist name that do not match any record in the `Artist` table, the `SELECT` check returns no rows. The application

does not insert anything and displays an error message. The employee is then returned to the menu.

Cancel an Event

Successful removal:

```
=====
      TiQ Employee Menu
=====
  1. Add a new event
  2. Cancel an event
  3. Modify ticket prices
  0. Back
=====
Enter your choice: 2

--- Cancel an Event ---
Tour name: Brass & Boulevard
Event date (YYYY-MM-DD): 2025-05-28
Management license number: 8360087317
Artist name: Tampflex
Event cancelled successfully.
Ticket purchases removed : 1
Tickets removed           : 1
```

The employee enters the tour name, event date, license number, and artist name identifying the event to cancel. Behind the scenes the application first runs a `SELECT` on `Tour_Event` to confirm the event exists. If found, it proceeds to delete all dependent rows in the correct foreign key order: `Buys_Ticket` first (ticket purchase records), then `Ticket` (the seats themselves), then `Listing`, then `Concert` or `Fansign` depending on the event type, and finally the `Tour_Event` row itself. This order is important because deleting `Tour_Event` first would violate the foreign key constraints of all the other tables. The application reports how many ticket purchases and tickets were removed, giving the employee clear feedback on the impact of the cancellation.

Error example:


```

=====
      TiQ Employee Menu
=====
1. Add a new event
2. Cancel an event
3. Modify ticket prices
0. Back
=====
Enter your choice: 2

--- Cancel an Event ---
Tour name: Faux Event
Event date (YYYY-MM-DD): 2025-12-12
Management license number: 123456
Artist name: John Doe
Error: No matching event found. Nothing was cancelled.

```

When the employee enters details that do not match any existing event, the initial `SELECT` on `Tour_Event` returns no rows. The application displays an error message. No deletions are made.

Modify Ticket Prices

Successful modification:

```

=====
      TiQ Employee Menu
=====
1. Add a new event
2. Cancel an event
3. Modify ticket prices
0. Back
=====
Enter your choice: 3

--- Modify Ticket Prices ---
Tour name: Prism Pop
Event date (YYYY-MM-DD): 2025-06-22
Management license number: 8578803496
Artist name: Stringtough

Current tickets for this event:
  Section      Tier      Price      Seats
-----
  18           GA       597.00      1
  51           VIP       686.00      1

Update prices for:
T - A specific tier
S - A specific section
A - All tickets for this event
Choice: T
New price: 500
Tier name: GA
Price updated to 500.00. 1 ticket(s) affected.

```

The employee identifies the event by tour name, date, license number, and artist name. The application queries the `Ticket` table and displays a grouped summary of all current sections, tiers, prices, and seat counts for that event, giving the

employee a clear picture before making changes. The employee then chooses to update by tier (T), section (S), or all tickets (A). In the example shown, the employee selects T for tier, enters a new price of 500, and specifies the GA tier. The application runs an `UPDATE` on `Ticket` filtered by the event fields and `tier = 'GA'`, and displays a confirmation message.

Error example:

```
=====
      TiQ Employee Menu
=====
1. Add a new event
2. Cancel an event
3. Modify ticket prices
0. Back
=====
Enter your choice: 3

--- Modify Ticket Prices ---
Tour name: Prism Pop
Event date (YYYY-MM-DD): 2025-06-22
Management license number: 8578803496
Artist name: Stringtough

Current tickets for this event:
Section      Tier      Price      Seats
-----
18           GA       500.00     1
51           VIP       686.00     1

Update prices for:
T - A specific tier
S - A specific section
A - All tickets for this event
Choice: A
New price: -100
Price cannot be negative. No changes made.
```

When the employee enters a negative price of -100, the application catches this before touching the database and displays an error message. The prices in the database remain unaffected.

Question 4 (part two: Fan Menu)

Landing Page

Upon selecting the fan menu, the user is prompted for an email. A valid fan email must be entered in order to proceed to fan menu options

```
○ cs421g111@winter2026-comp421:~$ java simpleJDBC
Connected to DB2.

TiQ Main Menu
    1. Employees
    2. Fans
    0. Exit
Please Enter Your Option: 2
Fan email: ld2@weather.com

=====
      TiQ Fan Menu
=====
1. Add artist to Likes
2. View liked artists
3. View events (filter)
4. Buy tickets
5. View purchased tickets
0. Back
=====
Please Enter Your Option: █
```

View Liked Artists

After choosing “view liked artists” option in the fan menu, a list of all artists that the fan likes is displayed in a table

```
Please Enter Your Option: 2

--- View Liked Artists ---

Liked artists:
This fan has no liked artists.
```

Add Artist to Likes

After choosing “Add Artist to Likes” option in the Fan Menu, the fan gets a list of questions to specify the specific Artist they want to add to their liked list.

The first filter is over the Artist name, then all artists with the same name is shown with their specific license number (since artists names are only unique under their Management license number). To view update, the fan can revisit option 2.

```
Please Enter Your Option: 1

--- Add Artist to Likes ---
Artist name: Y-Solowarm

Matching artists:
License: 6864278951 | Artist: Y-Solowarm
License: 945569336 | Artist: Y-Solowarm
Enter management license number: 6864278951
Artist added to Likes successfully.
```

```
Please Enter Your Option: 2

--- View Liked Artists ---

Liked artists:
License: 6864278951 | Artist: Y-Solowarm
```

If a fan already likes the artist, it is noted when the fan tries to add the artist to their liked list again:

```
Please Enter Your Option: 1
```

```
--- Add Artist to Likes ---
```

```
Artist name: Y-Solowarm
```

```
Matching artists:
```

```
License: 6864278951 | Artist: Y-Solowarm
```

```
License: 945569336 | Artist: Y-Solowarm
```

```
Enter management license number: 6864278951
```

```
Fan already likes this artist.
```

Also if no valid artist exists with the specific name or License number, the error is caught:

```
Please Enter Your Option: 1
```

```
--- Add Artist to Likes ---
```

```
Artist name:
```

```
Matching artists:
```

```
Error: No artist found with that name.
```

View events

After selecting “view events”, the fan has a series of questions used to filter the tickets for specific artist. The first filter is over the Artist name. The second and third filter is on the price limit with the second filter being the minimum price and the third filter being the maximum price. If any filter is left blank, it will include all available events. So, if Artist name is left blank, it will show all artists. If minimum and or maximum ticket price is left blank, it shows all ticket prices.

```

Please Enter Your Option: 3

--- View Events ---
Artist name (leave blank for all): Y-Solowarm
Minimum ticket price (leave blank for no minimum):
Maximum ticket price (leave blank for no maximum):

Available Tour Events:
License: 6864278951 | Artist: Y-Solowarm | Tour: Pure Resonance | Date: 2025-07-07 | Venue: Almo Stadium | Address: 4 Hayes Trail | Starting Price: $265.0
License: 6864278951 | Artist: Y-Solowarm | Tour: Pure Resonance | Date: 2025-08-25 | Venue: Lien Hall | Address: 9 Lukken Place | Starting Price: $450.0
License: 945569336 | Artist: Y-Solowarm | Tour: Pitch Perfected | Date: 2025-11-18 | Venue: Almo Stadium | Address: 4 Hayes Trail | Starting Price: $736.0
License: 945569336 | Artist: Y-Solowarm | Tour: Pitch Perfected | Date: 2025-11-19 | Venue: SHYP Center | Address: 55 Hoepker Center | Starting Price: $845.0

```

```

Please Enter Your Option: 3

--- View Events ---
Artist name (leave blank for all): Y-Solowarm
Minimum ticket price (leave blank for no minimum): 300
Maximum ticket price (leave blank for no maximum): 799

Available Tour Events:
License: 6864278951 | Artist: Y-Solowarm | Tour: Pure Resonance | Date: 2025-07-07 | Venue: Almo Stadium | Address: 4 Hayes Trail | Starting Price: $327.0
License: 6864278951 | Artist: Y-Solowarm | Tour: Pure Resonance | Date: 2025-08-25 | Venue: Lien Hall | Address: 9 Lukken Place | Starting Price: $450.0
License: 945569336 | Artist: Y-Solowarm | Tour: Pitch Perfected | Date: 2025-11-18 | Venue: Almo Stadium | Address: 4 Hayes Trail | Starting Price: $736.0

```

Buying tickets

After choosing the “Buy Tickets” option in the fan menu, a series of questions are used to filter down to a ticket, which a fan can choose to purchase.

```

=====
Please Enter Your Option: 4

--- Buy Ticket ---
Artist Management license number: 7623975961
Artist name: Vagran
Tour name: The Improvisation
Tickets for upcoming events for Vagran:
| Artist: Vagran | Tour: The Improvisation | Date: 2025-05-24 | Venue: Everett Stadium | Address: 6410 New Castle Junction | Section: 12 | Seat: 121 | Price: 6767.00 | Tier: VIP
| Artist: Vagran | Tour: The Improvisation | Date: 2025-05-24 | Venue: Everett Stadium | Address: 6410 New Castle Junction | Section: 94 | Seat: 182 | Price: 6767.00 | Tier: VIP
| Artist: Vagran | Tour: The Improvisation | Date: 2025-05-24 | Venue: Everett Stadium | Address: 6410 New Castle Junction | Section: 25 | Seat: 32 | Price: 306.00 | Tier: GA
| Artist: Vagran | Tour: The Improvisation | Date: 2025-05-24 | Venue: Everett Stadium | Address: 6410 New Castle Junction | Section: 204 | Seat: 25 | Price: 6767.00 | Tier: VIP
Venue name: Everett Stadium
Date (YYYY-MM-DD): 2025-05-24
Tickets for upcoming events for Vagran at Everett Stadium on 2025-05-24:
| Artist: Vagran | Tour: The Improvisation | Date: 2025-05-24 | Venue: Everett Stadium | Address: 6410 New Castle Junction | Section: 12 | Seat: 121 | Price: 6767.00 | Tier: VIP
| Artist: Vagran | Tour: The Improvisation | Date: 2025-05-24 | Venue: Everett Stadium | Address: 6410 New Castle Junction | Section: 204 | Seat: 25 | Price: 6767.00 | Tier: VIP
| Artist: Vagran | Tour: The Improvisation | Date: 2025-05-24 | Venue: Everett Stadium | Address: 6410 New Castle Junction | Section: 25 | Seat: 32 | Price: 306.00 | Tier: GA
| Artist: Vagran | Tour: The Improvisation | Date: 2025-05-24 | Venue: Everett Stadium | Address: 6410 New Castle Junction | Section: 94 | Seat: 182 | Price: 6767.00 | Tier: VIP
Section number: 12
Seat number: 121
Ticket for section 12, seat 121 for Vagran at Everett Stadium on 2025-05-24:
| Artist: Vagran | Tour: The Improvisation | Date: 2025-05-24 | Venue: Everett Stadium | Address: 6410 New Castle Junction | Section: 12 | Seat: 121 | Price: 6767.00 | Tier: VIP
Would you like to purchase this ticket? (y/n): y
How would you like to pay? (credit/debit): credit
Purchase successful of Vagran: The Improvisation on 2025-05-24 at Everett Stadium, section 12 seat 121

```

The first filter is over an Artist’s Management License Number (since artist names are only unique under their Management), Artist name and Tour name. The second filters over Venue and Event date. The last filters over section and seat, which brings it down to one ticket, where the Fan can agree to purchase with credit/debit.

If at any point no entries are found, invalid input is entered, or a database error occurs, the user is sent back to the main menu. The user is also sent back to the main menu if they do not purchase the ticket. shown in an example here:

```
=====
Please Enter Your Option: 4

--- Buy Ticket ---
Artist Management license number: 3
Artist name: 3
Tour name: 3
No events found for 3. Try again?

=====
TiQ Fan Menu
=====
1. Add artist to Likes
2. View liked artists
3. View events (filter)
4. Buy tickets
5. View purchased tickets
```

Viewing purchased tickets

After choosing the “View purchased tickets” option in the fan menu, all the tickets the Fan has purchased will be listed. For example, the ticket purchased in the “Buying tickets” sample screenshot is listed here.

```
=====
Please Enter Your Option: 5

Your purchased tickets:
| License number: 2877116198 | Artist: Voyatouch | Tour: Moonlight Sonata | Date: 2025-07-23 | Venue: Spenser Hill Stadium | Address: 525 Manitowish Hill | Section: 73 | Seat: 123 | Price: 1132.00 | Tier: VIP | Purchase Time: 2024-10-01 10:06:00.0 | Purchase Method: Credit
| License number: 7623975961 | Artist: Vagran | Tour: The Improvisation | Date: 2025-05-24 | Venue: Everett Stadium | Address: 6410 New Castle Junction | Section: 12 | Seat: 121 | Price: 6767.00 | Tier: VIP | Purchase Time: 2026-03-29 10:34:03.329 | Purchase Method: Credit
```

Section 5: Index

Execution:

```
db2 => CREATE INDEX idx_artist_industry ON Artist(industry, artist_name);
DB20000I The SQL command completed successfully.
```

Description:

This is a composite non-unique index on the `Artist` table, built on the columns `industry` and `artist_name` in that order. It is not built on a primary key or unique constraint, as the primary key of `Artist` is `(license_no, artist_name)`.

This index directly supports the 'Browsing Genres' feature of our application. Using this feature, fans can browse artists filtered by a genre (industry). This triggers a query of the form:

```
SELECT * FROM Artist WHERE industry = 'Pop';
```

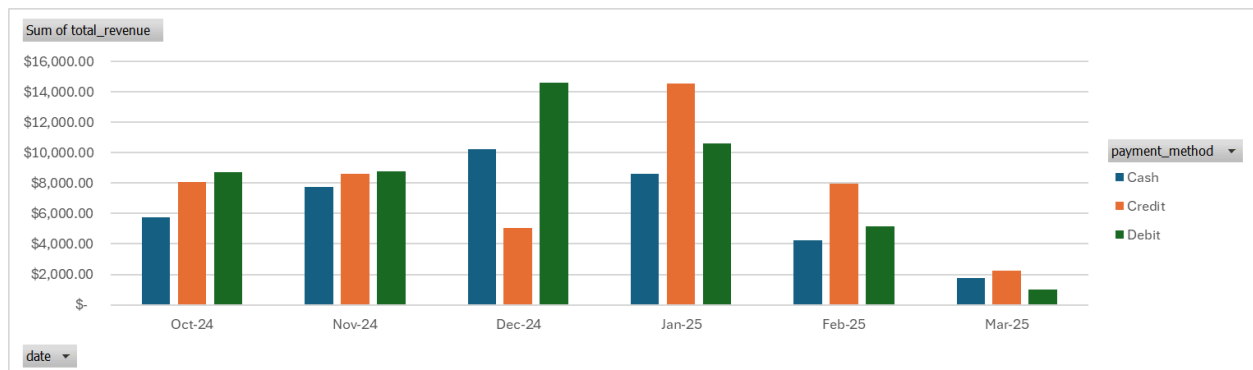
Without the index, DB2 would perform a full table scan of `Artist`, examining every row to find matches. With `idx_artist_industry`, since `industry` is the leftmost column, DB2 can jump directly to the relevant portion of the index and retrieve only the matching rows, which is significantly faster as the `Artist` table grows.

6. VISUALISATION

(i) SQL

```
EXPORT TO result.csv OF DEL MODIFIED BY NOCHARDEL
SELECT
    VARCHAR_FORMAT(B.purchase_time, 'YYYY-MM') AS year_month,
    B.purchase_method,
    COUNT(*) AS tickets_sold,
    DECIMAL(SUM(T.price), 10, 2) AS total_revenue
FROM Buys_Ticket B
JOIN Ticket T
    ON B.license_no = T.license_no
   AND B.artist_name = T.artist_name
   AND B.tour_name = T.tour_name
   AND B.event_date = T.event_date
   AND B.venue_address = T.venue_address
   AND B.section = T.section
   AND B.seat_no = T.seat_no
GROUP BY
    VARCHAR_FORMAT(B.purchase_time, 'YYYY-MM'),
    B.purchase_method
ORDER BY
    year_month,
    B.purchase_method
```

(ii) Chart



(iii) Explanation

This graph shows the **monthly total sales grossed by each payment method**, from October 2024 to March 2025. Recall the three valid payment methods on our application are *cash*, *debit* and *credit*. In business logic, this is pertinent to identify which period of the year is marked the most by sales, as well as analyzing the *Fans* purchasing trends. This could aid in decisional purposes (eg. should we remove cash option or is it still a popular means of

payment). The payment method is placed as a filter in the making of the pivot chart. The analyst can select a specific payment method to generate a chart for it.

Another interesting data would be to visualize the net revenue during the same time period, but all payment methods combined. The chart is left for reference in the third sheet of *Q6_Visualization_Excel*.

7. CREATIVITY

(a) Description

We created a **trigger** that serves to **audit the ticket price modifications** that have been made. This gives out all the details about the ticket that had its price modified. It notes down which employee made the change, which ticket it affected, what time the change was made and all the additional pertinent ticket information.

(b) Program code of the trigger

```
--#SET TERMINATOR @

CREATE VARIABLE CURRENT_EMPLOYEE_ID INTEGER DEFAULT NULL
@

CREATE TABLE Ticket_Price_Audit (
    audit_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    license_no VARCHAR(15) NOT NULL,
    artist_name VARCHAR(50) NOT NULL,
    tour_name VARCHAR(50) NOT NULL,
    event_date DATE NOT NULL,
    venue_address VARCHAR(125) NOT NULL,
    section VARCHAR(10) NOT NULL,
    seat_no INTEGER NOT NULL,
    old_price DECIMAL(10,2),
    new_price DECIMAL(10,2),
    changed_by_employee_id INTEGER,
    changed_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
)
@

CREATE TRIGGER TRG_TICKET_PRICE_AUDIT
AFTER UPDATE OF price ON Ticket
REFERENCING OLD AS O NEW AS N
FOR EACH ROW
WHEN (O.price IS DISTINCT FROM N.price)
BEGIN ATOMIC
    INSERT INTO Ticket_Price_Audit (
        license_no, artist_name, tour_name, event_date,
venue_address,section,
        seat_no, old_price, new_price,changed_by_employee_id,changed_at
    )
    VALUES (
        O.license_no, O.artist_name, O.tour_name, O.event_date,
O.venue_address,
        O.section, O.seat_no, O.price, N.price, CURRENT_EMPLOYEE_ID,
CURRENT_TIMESTAMP
    );
END
@
```

(c) Execution

First, Employee **6** modify ticket price of a VIP tier for a concert

```
TiQ Main Menu
1. Employees
2. Fans
0. Exit
Please Enter Your Option: 1
Employee ID: 6

=====
TiQ Employee Menu
=====
1. Add a new event
2. Cancel an event
3. Modify ticket prices
0. Back
=====
Enter your choice: 3

--- Modify Ticket Prices ---
Tour name: The Improvisation
Event date (YYYY-MM-DD): 2025-05-24
Management license number: 7623975961
Artist name: Vagram

Current tickets for this event:
Section      Tier      Price      Seats
-----
12           VIP      674.00     1
134          GA       312.00     1
176          VIP      652.00     1
185          VIP      1052.00    1
204          VIP      1287.00    1
25           GA       306.00     1
73           GA       389.00     1
94           VIP      1036.00    1
99           VIP      1256.00    1

Update prices for:
T - A specific tier
S - A specific section
A - All tickets for this event
Choice: T
New price: 6767
Tier name: VIP
Price updated to 6767.00. 6 ticket(s) affected.
```

This ticket modification for a Section did not pass. Employee **6** entered a tier instead of a valid Section.

```
=====
TiQ Employee Menu
=====
1. Add a new event
2. Cancel an event
3. Modify ticket prices
0. Back
=====
Enter your choice: 3

--- Modify Ticket Prices ---
Tour name: Electric Funeral
Event date (YYYY-MM-DD): 2025-05-24
Management license number: 5848943849
Artist name: Fix San

Current tickets for this event:
Section      Tier      Price      Seats
-----
26           VIP      1072.00    1
29           GA       554.00     1
35           VIP      886.00     1
38           GA       92.00      1
42           VIP      1247.00    1
53           GA       222.00     1
58           VIP      918.00     1

Update prices for:
T - A specific tier
S - A specific section
A - All tickets for this event
Choice: S
New price: 420
Section name: GA
Price updated to 420.00. 0 ticket(s) affected.
```

Employee **5** successfully updates the ticket for the GA tier of this concert.

```
=====
TiQ Employee Menu
=====
1. Add a new event
2. Cancel an event
3. Modify ticket prices
0. Back
=====
Enter your choice: 3

--- Modify Ticket Prices ---
Tour name: Electric Funeral
Event date (YYYY-MM-DD): 2025-05-24
Management license number: 5848943849
Artist name: Fix San

Current tickets for this event:
Section      Tier      Price      Seats
-----
26           VIP      1072.00    1
29           GA       554.00     1
35           VIP      886.00     1
38           GA       92.00      1
42           VIP      1247.00    1
53           GA       222.00     1
58           VIP      918.00     1

Update prices for:
T - A specific tier
S - A specific section
A - All tickets for this event
Choice: T
New price: 606
Tier name: GA
Price updated to 606.00. 3 ticket(s) affected.
```

cont'd

Viewing the **TICKET_PRICE_AUDIT** table:

(**SELECT * FROM TICKET_PRICE_AUDIT;**) :

AUDIT_ID	LICENSE_NO	ARTIST_NAME	TOUR_NAME	EVENT_DATE	VENUE_ADDRESS	SECTION	SEAT_NO	OLD_PRICE	NEW_PRICE	CHANGED_BY_EMPLOYEE_ID	CHANGED_AT
1	7623975961	Vagran	The Improvisation	08/24/2025	6418 New Castle Junction	12	121	676.00	6767.00	6	2026-03-28-21.37.08.109683
2	7623975961	Vagran	The Improvisation	08/24/2025	6418 New Castle Junction	176	61	652.00	6767.00	6	2026-03-28-21.37.08.109683
3	7623975961	Vagran	The Improvisation	08/24/2025	6418 New Castle Junction	185	100	1052.00	6767.00	6	2026-03-28-21.37.08.109683
4	7623975961	Vagran	The Improvisation	08/24/2025	6418 New Castle Junction	204	25	1287.00	6767.00	6	2026-03-28-21.37.08.109683
5	7623975961	Vagran	The Improvisation	08/24/2025	6418 New Castle Junction	94	182	1036.00	6767.00	6	2026-03-28-21.37.08.109683
6	7623975961	Vagran	The Improvisation	08/24/2025	6418 New Castle Junction	99	53	1256.00	6767.00	6	2026-03-28-21.37.08.109683
7	5808943849	Flx San	Electric Funeral	05/24/2025	993 Declaration Junction	29	188	556.00	606.00	5	2026-03-28-21.44.56.896879
8	5808943849	Flx San	Electric Funeral	05/24/2025	993 Declaration Junction	38	141	92.00	606.00	5	2026-03-28-21.44.56.896879
9	5808943849	Flx San	Electric Funeral	05/24/2025	993 Declaration Junction	53	167	222.00	606.00	5	2026-03-28-21.44.56.896879

Running a more specific query to check the modified tickets with only pertinent informations:

```
SELECT changed_by_employee_id, old_price, new_price, changed_at,
       tour_name, event_date, section, seat_no
FROM Ticket_Price_Audit
ORDER BY changed_at DESC
FETCH FIRST 20 ROWS ONLY;
```

Notice how Employee 6 modifications for Electric Funeral were invalid, so did not pass onto the table. But Employee 5 valid modifications were recorded.

```
cs421g111@winter2026-comp421:~$ db2 "SELECT changed_by_employee_id, old_price, new_price, changed_at,
       tour_name, event_date, section, seat_no
FROM Ticket_Price_Audit
ORDER BY changed_at DESC
FETCH FIRST 20 ROWS ONLY"
```

CHANGED_BY_EMPLOYEE_ID	OLD_PRICE	NEW_PRICE	CHANGED_AT	TOUR_NAME	EVENT_DATE	SECTION	SEAT_NO
-	929.00	900.00	2026-03-28-21.59.55.091036	Pure Resonance	08/25/2025	6	109
-	821.00	900.00	2026-03-28-21.59.55.090967	Pure Resonance	08/25/2025	15	117
-	1101.00	900.00	2026-03-28-21.59.55.090889	Backroad Ballads	08/25/2025	21	149
-	1029.00	900.00	2026-03-28-21.59.55.090832	Backroad Ballads	08/25/2025	20	68
-	53.00	58.30	2026-03-28-21.59.55.090783	Backroad Ballads	08/25/2025	18	79
-	770.00	847.00	2026-03-28-21.59.55.090726	Backroad Ballads	08/25/2025	9	188
-	1060.00	900.00	2026-03-28-21.59.55.090623	Pure Resonance	08/25/2025	12	18
-	302.00	332.20	2026-03-28-21.59.55.088061	Backroad Ballads	08/25/2025	16	108
5	554.00	606.00	2026-03-28-21.44.56.896879	Electric Funeral	05/24/2025	29	188
5	222.00	606.00	2026-03-28-21.44.56.896879	Electric Funeral	05/24/2025	53	167
5	92.00	606.00	2026-03-28-21.44.56.896879	Electric Funeral	05/24/2025	38	141
6	674.00	6767.00	2026-03-28-21.37.08.109683	The Improvisation	05/24/2025	12	121
6	1256.00	6767.00	2026-03-28-21.37.08.109683	The Improvisation	05/24/2025	99	53
6	1036.00	6767.00	2026-03-28-21.37.08.109683	The Improvisation	05/24/2025	94	182
6	1287.00	6767.00	2026-03-28-21.37.08.109683	The Improvisation	05/24/2025	204	25
6	1052.00	6767.00	2026-03-28-21.37.08.109683	The Improvisation	05/24/2025	185	100
6	652.00	6767.00	2026-03-28-21.37.08.109683	The Improvisation	05/24/2025	176	61

17 record(s) selected.

(Please disregard the modifications with no employee ID – a teammate was testing the modifying ticket price functions before I could take a clean screenshot. They got **audited**. Refer to the attribute **changed_at** times timestamp for proof!)

8. WORKING TOGETHER

We maintained our group collaboration method that we adopted throughout P1 to P2. That is, we had an initial meeting to discuss what P3 consists of, what are its biggest challenges, and clarify any questions and concerns. We also set ourselves a team deadline to complete P3, prior to the class due date so that we could come together and discuss our work. Then we assigned the questions to one another. Since question 4 was the biggest part of P3, it only made sense that multiple teammates tackled it. We then started our part on our own and maintained communication. Many features implemented throughout P3 are intertwined. As a result, we had constant exchanges with one another. We helped look through each others' code to assist in debugging. We did have to slightly push back our team due date since some teammates could not finish their part in time due to their other courses commitment but this only showed that setting a team due date serves its positive purpose. We finally met again to test out together the application, read throughout P3 submission files and discuss the presentation. We will have another meeting prior to the presentation to separate how the content will be covered. Overall, we worked really great together.